Dynamo Pay

Merchant & API User manual

Your guide to collecting and managing UPI payments

Set up your account, connect your website, and manage payments from the Dynamo Pay Merchant Console. Each section explains the action to take and the result to expect.

Version 1.1 | 31 October 2026

Before you begin

Have your gateway address, merchant credentials and receiving UPI ID ready. Your private account handover contains the access details. You can also find your integration credentials under Developer tools in the console.

Use this manual with your development team. The separate private credential sheet is for the account owner and authorised server administrators.

What you will find inside

2 Setup and authentication
3 Create a UPI payment
4 Follow the payment status
5 Receive and verify callbacks
6 IFSC lookup and wallet balances
7 UPI controls and reconciliation
8 Daily operations and troubleshooting
9 Credentials and account access
10 Successful checks and common questions

Setup and authentication

Your backend talks to Dynamo Pay. Your customer opens the checkout page returned by the API. Keep that separation: API secrets belong on your server, never in browser JavaScript or a mobile app bundle.

Your gateway address

```
DYNAMO_BASE_URL=https://your-assigned-gateway.example
```

Use the HTTPS origin supplied for your deployment. A temporary tunnel address changes when the tunnel restarts. Give customers a fixed HTTPS domain for ongoing integrations.

Send these headers on every API request

Header	What to send
Client-ID	Your Dynamo Pay client ID
Client-Secret	Your backend client secret
Service-Id	Your assigned merchant service ID
Content-Type	application/json

Use the same Service-Id for your own collection, status, IFSC and wallet requests. Do not copy service IDs from somebody else's examples.

Available endpoints

Function	Method	Endpoint
Create UPI payment	POST	/v1.0/collect/upi
Check status	POST	/v1.0/payin/checkstatus
Look up IFSC	POST	/v1.0/ifsc/{ifsc}
Read wallet balances	POST	/v1.0/wallet
Save callback / IP list	POST	/v1.0/merchant/webhook

Server IP restrictions

You can allow only specific server IPs to use your credentials. Enter your backend's public outbound IP address, not your laptop's private address. Exact IPv4 and IPv6 addresses are supported; CIDR ranges are not. An empty list leaves IP restriction off.

The receiving-UPI allowlist is a separate setting. It controls where payments go, rather than which server can call the API.

Create a UPI payment

Create one transaction ID for each merchant order and save it in your database before making the request. This gives you a reliable way to check the result if your connection drops.

```
POST /v1.0/collect/upi
```

```
{
  "amount": "201.00",
  "uniquetransactionid": "ORDER-1001",
  "remarks": "Digital service",
  "name": "Customer name",
  "emailid": "customer@example.com",
  "mobileno": "9876543210"
}
```

Required: amount and uniquetransactionid. The transaction ID accepts 1-120 letters, digits, hyphens or underscores. Customer details and remarks are optional.

Send amounts in rupees with no more than two decimal places. With the current merchant settings, new UPI payments must be between INR 200.00 and INR 20,000.00. These limits can be changed in the console. The versioned API itself has a floor of INR 100.00; the higher configured minimum takes precedence.

What comes back

```
{
  "transaction": "payment-uuid",
  "uniquetransactionid": "ORDER-1001",
  "amount": "201.00",
  "currency": "INR",
  "status": "PENDING",
  "internalStatus": "ACTIVE",
  "checkoutUrl": "https://your-gateway/checkout/payment-uuid",
  "qr": "https://your-gateway/checkout/payment-uuid/qr.svg",
  "intent": "upi://pay?pa=merchant%40bank&am=201.00&cu=INR"
}
```

This response is shortened for readability. It also includes creation time, expiry when available and payee details. Examples in this manual are illustrative.

Redirect the customer to checkoutUrl. It opens the Dynamo Pay QR and UPI app choices directly. The qr field is an SVG image URL; intent is a UPI app link. For queued payments both are null until activation. Never display an old or expired QR.

A successful API request creates an order. It does not mean the customer has paid.

Follow the payment status

```
POST /v1.0/payin/checkstatus
```

```
{ "uniquetransactionid": "ORDER-1001" }
```

You can also send transaction with the payment UUID. Use your merchant credentials; you cannot query another merchant's payment.

Public status	What it means	What your app should do
PENDING	Payment is queued, active, being checked or awaiting bank verification.	Keep the order unpaid. Show a clear waiting message.
SUCCESS	Receipt has been confirmed.	Verify order, amount, currency and method before fulfilment.
FAILED	Payment failed, expired or was cancelled.	Show the specific outcome from internalStatus. Avoid asking a debited customer to pay again.

When a customer taps Payment done

Dynamo Pay starts checking for a matching bank notification. The button itself does not prove receipt. If the notification has not arrived when the short verification timer ends, the order stays VERIFICATION_PENDING, not automatically failed.

A customer can save or correct the 12-digit UPI reference while waiting. This is a reported reference, not proof. A matching bank record is still needed for automatic confirmation. Poll status no faster than once every five seconds and stop at a final result.

Know the source of confirmation

confirmationSource identifies BANK_OBSERVER or MERCHANT when available. A merchant-confirmed receipt is recorded separately from automatic bank verification. Missing bank references remain null; a typed reportedUtr is never presented as a verified bank reference.

Repeat requests safely

The same transaction ID and amount returns the existing order with HTTP 200. A new order returns HTTP 201. Reusing an ID with a different amount returns HTTP 409. If a request times out, retry with the same ID rather than creating another charge.

A final order keeps its ID. If the customer deliberately starts a new attempt, use a new merchant transaction ID. The normal active checkout window is five minutes after activation; time in the queue is separate.

Receive and verify callbacks

Give Dynamo Pay a public HTTPS URL on your server. When a payment reaches a final outcome, Dynamo Pay sends a JSON POST to that URL. Save the event reliably, process it once and return HTTP 200.

```
POST /v1.0/merchant/webhook

{
  "webhookUrl": "https://your-store.example/payment-callback",
  "ipWhitelist": ["203.0.113.10"]
}
```

The IP above is an example. Replace it with your server's actual public IP, or omit the field to keep the existing list.

Check the signature before reading the event

The X-Gateway-Signature header contains sha256= followed by an HMAC-SHA256 digest. Calculate that digest from the original request bytes, using your webhook signing secret. Compare it in constant time. Do not parse and re-serialize JSON before checking it.

```
import crypto from 'node:crypto';

const expected = 'sha256=' + crypto
  .createHmac('sha256', process.env.DYNAMO_WEBHOOK_SECRET)
  .update(rawBody)
  .digest('hex');
const supplied = String(req.headers['x-gateway-signature'] || '');
const valid = supplied.length === expected.length &&
  crypto.timingSafeEqual(Buffer.from(supplied),
                         Buffer.from(expected));
if (!valid) { res.statusCode = 401; res.end(); return; }
const event = JSON.parse(rawBody.toString('utf8'));
// Validate, deduplicate and persist before acknowledging.
res.statusCode = 200;
res.end('OK');
```

Check the business details too

Match orderId, amountPaise, INR currency and payment method to your stored order. Only fulfil prepaid goods for a verified, eligible SUCCESS event. COD SUCCESS means an order was placed; cash has not been collected yet. Deduplicate X-Gateway-Delivery-Id and protect fulfilment with your own unique order constraint.

Callbacks include event, status, transaction/paymentId, uniquetransactionid/orderId, amount, amountPaise, currency, method, bankReferenceNumber/utr, senderVpa, payeeVpa, beneficiary, confirmationSource, completedAt and metadata. Fields without verified data can be null.

IFSC lookup and wallet balances

Look up an IFSC code

```
POST /v1.0/ifsc/SBIN0000001
Content-Type: application/json

{}
```

Send the same three authentication headers. An IFSC has four letters, a zero and six letters or digits. The response describes the bank and branch, including city, state, address, contact and supported payment networks.

```
{
  "ifsc": "SBIN0000001",
  "bank": "Bank name",
  "branch": "Branch name",
  "city": "City",
  "upi": true,
  "neft": true,
  "rtgs": true,
  "imps": true
}
```

This is a bank-directory lookup, not account ownership verification or a promise that a transfer will work. The directory is external. A lookup failure alone is not enough to declare a customer's bank account invalid.

Read the merchant wallet

```
POST /v1.0/wallet
Content-Type: application/json

{}
```

Field	Meaning
settledwallet	Settled amount recorded in the ledger.
unsettledbalance	Collections recorded as not yet settled.
lienbalance	Amount held or in transit for payouts.
loanbalance	Loan balance represented by the ledger.

All four balances are decimal INR strings, for example "1250.00". The response also identifies the merchant, currency and response time.

These are accounting balances, not a live bank-balance enquiry. A direct UPI credit to your bank does not automatically become money in a provider wallet. Reconcile bank and ledger records before making settlement decisions.

UPI controls and reconciliation

Open Merchant Console > UPI Accounts to manage the channel. Save changes when you are ready to apply them to new payment requests.

Control	How it works
Enable UPI payments	On accepts new UPI requests. Off rejects new requests. Existing payments can still be checked and reconciled.
Minimum amount	Currently INR 200.00. Requests below the saved minimum are rejected.
Maximum amount	Currently INR 20,000.00. Requests above the saved maximum are rejected.
Allowed receiving UPI IDs	Only these beneficiary IDs can receive new collections. This is not a list of customer payer IDs.

Separate allowed receiving IDs with commas or new lines. Leave the list empty to allow any receiving account configured for your merchant. Check the active receiving account before adding restrictions, or you may block your own collections.

Connect a bank confirmation channel

Android SMS: install the observer on the phone receiving bank credit messages. Grant the required permissions, scan the pairing QR from Alerts & observers, and keep the background service running.

Gmail: enter your Google OAuth client settings in Alerts & observers. Register the displayed callback URI, connect the inbox that receives bank credit emails and run the connection test. The account owner must authorize access.

If the customer paid but the order is pending

Do not ask them to pay again. Compare the bank credit with the receiving UPI account, amount and 12-digit reference. Let the observer match the real bank record. If more than one same-amount order could match, hold the credit for review instead of assigning it to the newest order.

An authorized merchant can confirm a receipt after checking the bank. That decision must remain labelled merchant-confirmed in the audit trail. It must not create a fictional bank observation.

Refunds and recipient splits

Refund requests, platform fees and recipient allocations can be tracked in the console. A recorded allocation or refund request does not send funds. Outgoing transfers need bank action or a separately configured provider. Save the actual transfer reference after the money is sent.

Daily operations and troubleshooting

Before you share your payment link

1. Use a stable HTTPS gateway address.
2. Store API and signing secrets only on your backend.
3. Confirm the receiving UPI ID and channel limits.
4. Connect and test a bank observer.
5. Verify signed callbacks and exact order amounts.
6. Test repeated requests, queued payments, cancellations and delayed credits.
7. Check the first real bank receipt and dashboard result together.

Callback delivery

Each delivery has a ten-second timeout and up to five total attempts. Any non-2xx response or network error is retried. The waits after an unsuccessful attempt are 5 seconds, 30 seconds, 2 minutes and 10 minutes, plus scheduler delay. Delivery ID and payload stay the same across retries.

The signature covers the payload, not the timestamp header. Keep durable delivery-ID and order deduplication; a timestamp check alone does not stop a replay. If delivery attempts run out, reconcile through the status API and your merchant operations process.

Common responses

HTTP	What to check
400	Amount, field format, channel toggle or receiving-UPI allowlist.
401	Client-ID, Client-Secret and Service-Id.
403	Your server IP allowlist or merchant ownership.
404	Transaction ID, IFSC or endpoint path.
409	Same order ID with a different amount, or an inactive QR.
429	Honour Retry-After. The configured default is 1,000 requests per IP per 60-second window.
5xx	Connection, provider or gateway configuration. Keep the same order ID when retrying an uncertain creation request.

A UPI app does not open

On a laptop, scan the QR with your phone. On a phone, choose an installed UPI app and return to checkout after paying. App launching depends on the device, browser, installed apps and bank acceptance. Keep the QR option available.

Keep this manual with your integration records. For a payment issue, note the merchant order ID, Dynamo Pay transaction ID, amount, receiving UPI ID, bank reference and approximate payment time. Never share an OTP, UPI PIN or client secret.

Credentials and account access

Your console login and API credentials serve different purposes. Use the email and password to manage the business. Use the merchant API credentials on your server to create payments and read their status.

Credential	Purpose
Console email and password	Sign in to the Merchant Console. Supplied in your private account handover.
Client-ID	Identifies your merchant when calling the payment API.
Client-Secret	Authenticates your server requests. Send it in the Client-Secret header.
Service-Id	Your assigned merchant identifier; use the same value across the documented endpoints.
Publishable key	Identifies the merchant in a supported browser checkout integration.
Webhook signing secret	Verifies payment callback signatures on your server.
Webhook URL	Your public HTTPS endpoint for receiving payment updates.

Find your keys

Sign in, open Developer tools, and copy the credentials into your server environment. Keep the values together under the correct merchant account. Confirm the callback URL before accepting customer payments.

```
DYNAMO_BASE_URL=https://your-assigned-gateway.example
DYNAMO_CLIENT_ID=your-client-id
DYNAMO_CLIENT_SECRET=your-client-secret
DYNAMO_SERVICE_ID=your-service-id
DYNAMO_WEBHOOK_SECRET=your-webhook-signing-secret
```

Example authenticated request

```
curl -X POST "$DYNAMO_BASE_URL/v1.0/wallet" \
  -H "Client-ID: $DYNAMO_CLIENT_ID" \
  -H "Client-Secret: $DYNAMO_CLIENT_SECRET" \
  -H "Service-Id: $DYNAMO_SERVICE_ID" \
  -H "Content-Type: application/json" \
  -d '{}'
```

Place secret values in your backend environment. Share only the public manual and publishable integration details with frontend developers. Internal administrator and observer tokens belong to the service configuration.

Successful checks and common questions

The completed automated checks below passed on 31 October 2026. The detailed test report and route inventory are available from the Developer API page.

Check	Result
331 automated tests across 42 suites	Successful
183 registered routes: access and invalid-input smoke checks	Successful
200 unique orders with queue scheduling	Successful
50 repeated requests resolved to one transaction	Successful
1,001 stored receiving accounts	Successful
10 merchants with parallel, matched simulated confirmations	Successful
Callback retries, signatures and attempt limits	Successful
Per-IP request limits and forwarding-header protection	Successful

How many payments can I accept at once?

Normal routing keeps one checkout active per receiving account and queues the next orders. The capacity check accepted all 200 test orders and completed their simulated confirmations. Ten separate merchants were also checked in parallel. These figures describe the tested application behaviour; bank completion time depends on actual payments and notification delivery.

Where should my team work each day?

Use Overview for collections and items needing attention, Transactions for payment details, and Refunds or Split payments for the related operations. Use UPI Accounts for receiving settings and Connections & alerts for SMS and Gmail.

What should I tell a customer who has already paid?

Ask for their order ID and bank reference. Check the receiving account and amount, then follow the pending payment through reconciliation. Confirm the result from the bank record before asking the customer to make another payment.

Can this manual be shared with customers?

Yes. Share this manual for setup and operating guidance. Provide each customer only their own assigned merchant credentials through a private handover.